

API Security Risk Analysis Report

Read-Only Security Assessment of Public Test APIs

Prepared by: Makuei Geu Makuei

Prepared for: Future Interns - Cyber Security Task 3 (2026)

Date: July 2026

APIs Tested: JSONPlaceholder (jsonplaceholder.typicode.com), ReqRes (reqres.in)

Testing Tool: Postman

1. Executive Summary

This report presents the findings of a read-only API security risk analysis performed against two publicly available test APIs: JSONPlaceholder and ReqRes. The purpose of this assessment was to evaluate common API security risks, including authentication enforcement, authorization controls, data exposure, input validation, and rate limiting, using the same non-destructive methodology a security consultancy would apply during a client API audit.

Six findings were documented across the two APIs. Three findings identify genuine risk areas (unauthenticated data exposure, broken object-level authorization, and missing rate limiting), while three findings confirm secure practices already in place (safe error handling on both APIs, and proper authentication enforcement with data minimization on ReqRes). Presenting both positive and negative findings reflects standard practice in professional security reporting and demonstrates that gaps were identified through systematic testing rather than assumption.

No exploitation, bypass, flooding, or denial-of-service testing was performed at any point. All testing was limited to safe, read-only GET requests against publicly documented demo endpoints, in line with the scope and ethics requirements of this task.

2. Scope & Methodology

2.1 Scope

- APIs tested: JSONPlaceholder (jsonplaceholder.typicode.com) and ReqRes (reqres.in) — both explicitly provided for public testing and learning.
- Testing limited to read-only requests (GET) against publicly documented endpoints.
- No exploitation, authentication bypass attempts, flooding/DoS testing, or testing of private/production systems was performed.

2.2 Methodology

Testing followed a structured, repeatable process for each API:

- Reviewed available API documentation to understand intended usage and authentication model.
- Sent baseline requests with no authentication to observe default access behavior.
- Where authentication was required, obtained a valid API key and re-tested with proper credentials.
- Inspected response bodies for the volume and sensitivity of data returned relative to the request.
- Tested invalid and edge-case inputs (non-existent IDs, non-numeric IDs) to assess input validation and error handling.
- Tested cross-resource requests (e.g., posts and comments tied to a specific user ID) to assess object-level authorization.
- Sent repeated rapid requests to check for the presence of rate limiting.
- Classified each finding by risk category, severity (Low / Medium / High), and business impact, and proposed remediation.

3. Tools Used

- Postman - API request construction, header/authentication testing, and response inspection.
- Browser - reviewing published API documentation.
- Google Docs / PDF - report documentation.

4. Findings Summary

The table below summarizes all findings. Full detail, business impact, and remediation guidance for each follows in Section 5.

#	API	Finding	Risk Category	Severity
1	JSONPlaceholder	Unauthenticated access to full user PII dataset	Open/Unauthenticated Endpoint; Excessive Data Exposure	Medium (High in production)
2	JSONPlaceholder	Safe error handling on invalid/non-existent IDs	Input Validation	Pass
3	JSONPlaceholder	No ownership checks on posts/comments (IDOR pattern)	Broken Object Level Authorization	Medium (High in production)
4	ReqRes	Proper authentication enforcement + data minimization	Authentication	Pass
5	ReqRes	Safe error handling on invalid/non-existent IDs	Input Validation	Pass

#	API	Finding	Risk Category	Severity
6	ReqRes	No rate limiting observed on repeated requests	Missing Rate Limiting	Medium

5. Detailed Findings

Finding 1: Unauthenticated Access to Full User PII Dataset

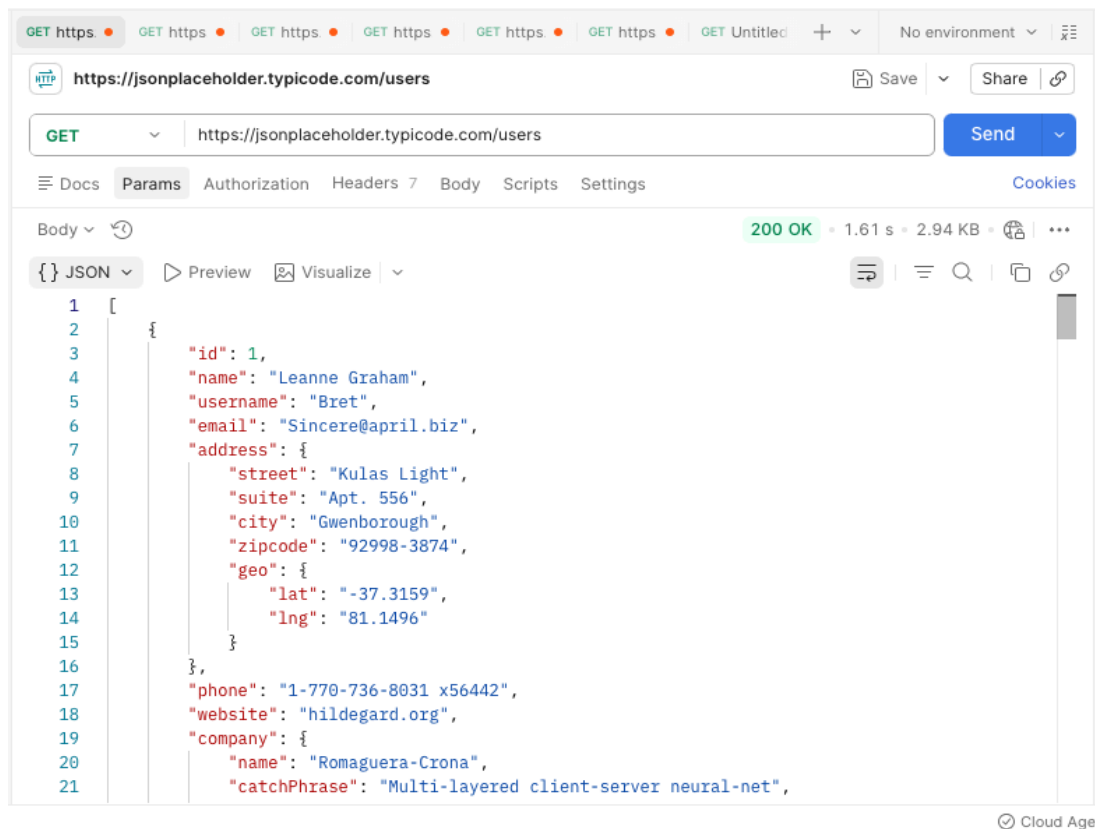
API Tested: JSONPlaceholder

Endpoint(s): GET /users

Test Performed: Sent a plain GET request with no headers, no API key, and no token.

Result: The endpoint returned all 10 user records in full, with no authentication challenge, no access restriction, and no rate limiting of any kind.

Data Exposed: Full name, username, email, complete physical address (street, suite, city, zip, and geo-coordinates), phone number, website, and employer details for every user.



Risk Category: Open/Unauthenticated Endpoint; Excessive Data Exposure

Severity: **Medium** (would be High in a production environment)

Business Impact: Any external party, including automated scrapers, could harvest the entire user database without authentication or detection, enabling phishing, identity theft, or social engineering campaigns against real users.

Remediation Suggestions:

- Require authentication (API key or OAuth token) on any endpoint that returns personally identifiable information.
- Apply field-level filtering so sensitive fields (address, geo-coordinates, phone) are only returned to clients with a legitimate need.
- Add rate limiting and monitoring to detect and slow bulk scraping attempts.

Finding 2: Safe Error Handling on Invalid and Non-Existent IDs

API Tested: JSONPlaceholder

Endpoint(s): GET /users/9999, GET /users/abc

Test Performed: Requested a non-existent numeric user ID and a non-numeric user ID to test input validation and error handling.

Result: Both requests returned a clean 404 Not Found with an empty JSON body. No stack traces, database errors, or internal implementation details were disclosed.

The image displays two screenshots of a REST client interface, likely Postman, showing the results of two GET requests to the JSONPlaceholder API.

Top Screenshot: The request is to `https://jsonplaceholder.typicode.com/users/abc` using the GET method. The response is a `404 Not Found` status, with a response time of 394 ms and a body size of 1.1 KB. The response body is an empty JSON object `{}`.

Bottom Screenshot: The request is to `https://jsonplaceholder.typicode.com/users/9999` using the GET method. The response is a `404 Not Found` status, with a response time of 389 ms and a body size of 1.09 KB. The response body is an empty JSON object `{}`.

Both screenshots show the REST client interface with tabs for Docs, Params, Authorization, Headers, Body, Scripts, and Settings. The response body is displayed in the JSON format.

Risk Category: Input Validation

Severity: Pass

Business Impact: None; this is a positive control. Generic, non-verbose error responses reduce the ability of an attacker to fingerprint backend technology or structure through reconnaissance.

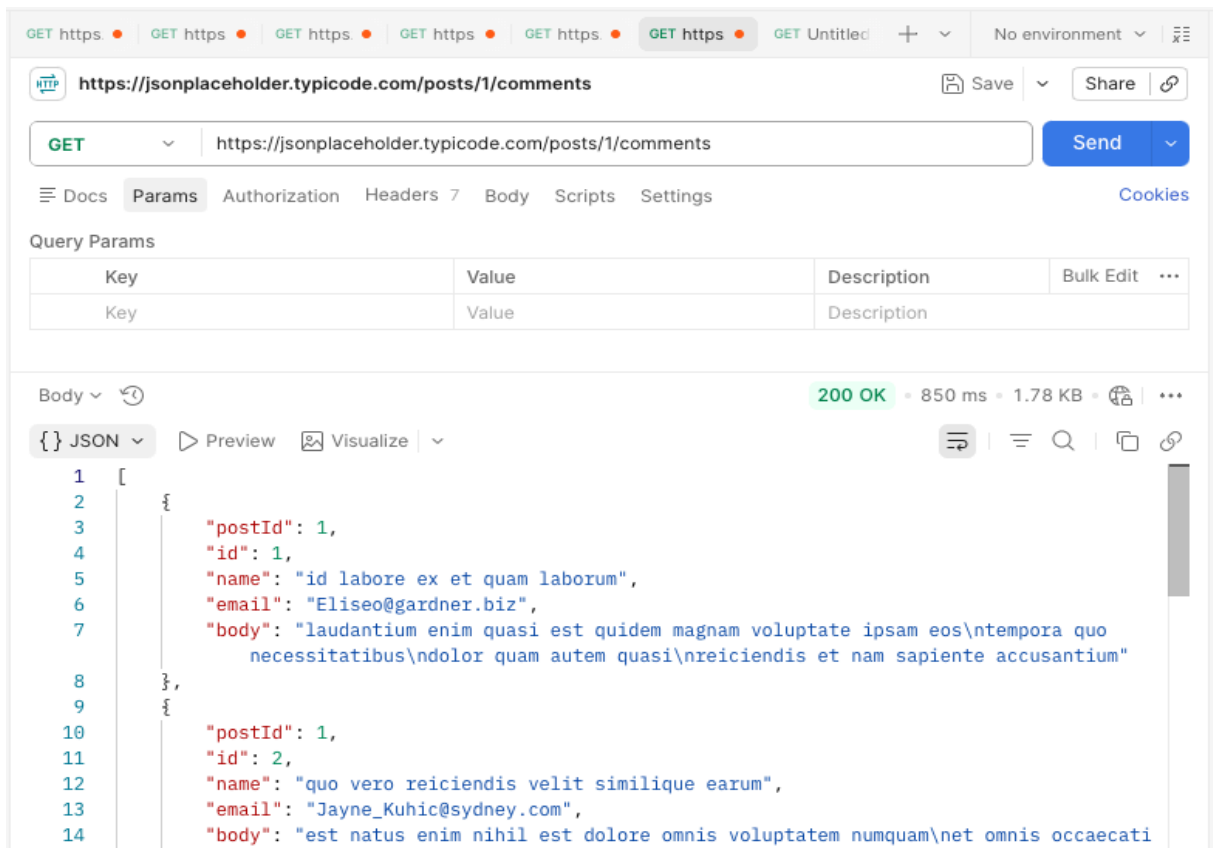
Finding 3: No Ownership Checks on Related Resources (IDOR Pattern)

API Tested: JSONPlaceholder

Endpoint(s): GET /posts?userId=1, GET /posts/1/comments

Test Performed: Queried resources by supplying a numeric ID/filter directly, with no session, token, or ownership verification of any kind.

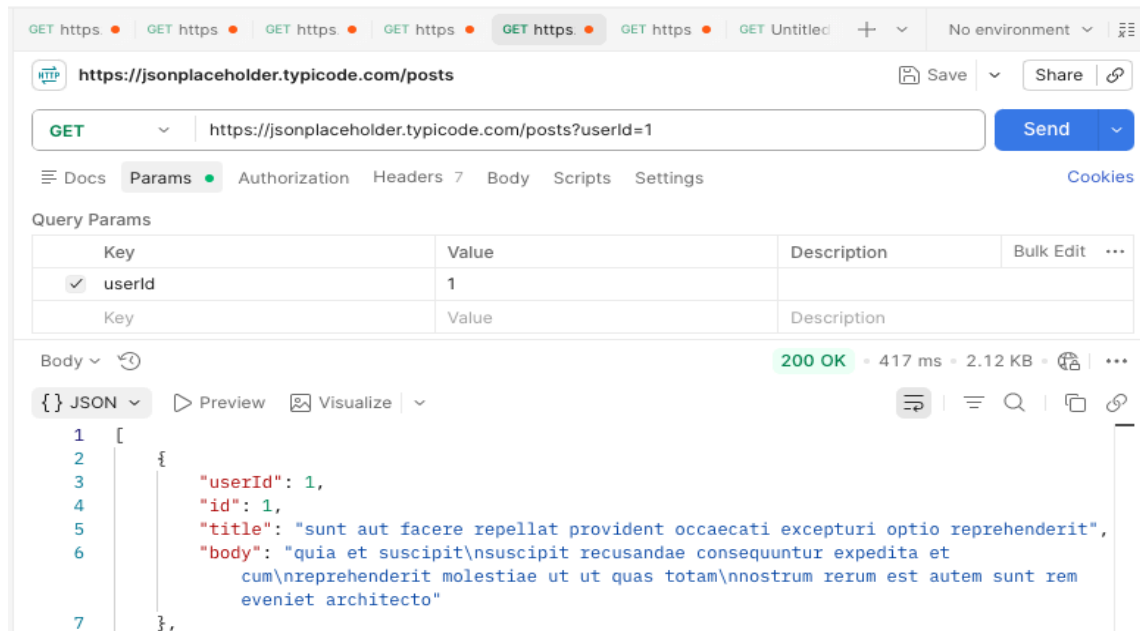
Result: Full data was returned for any ID requested. Changing the userId or post ID parameter would allow enumeration of every user's posts and every comment author's contact information, with no restriction.



The screenshot shows a REST client interface with the following details:

- URL:** `https://jsonplaceholder.typicode.com/posts/1/comments`
- Method:** GET
- Response Status:** 200 OK
- Response Time:** 850 ms
- Response Size:** 1.78 KB
- Response Body (JSON):**

```
[
  {
    "postId": 1,
    "id": 1,
    "name": "id labore ex et quam laborum",
    "email": "Eliseo@gardner.biz",
    "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
  },
  {
    "postId": 1,
    "id": 2,
    "name": "quo vero reiciendis velit similique earum",
    "email": "Jayne_Kuhic@sydney.com",
    "body": "est natus enim nihil est dolore omnis voluptatem numquam\net omnis occaecati"
  }
]
```



Data Exposed: Post titles/bodies tied to a specific user, plus commenter name and email address on every comment.

Risk Category: Broken Object Level Authorization (OWASP API1:2023); Excessive Data Exposure

Severity: **Medium** (*would be High in production , this is the #1 risk category on the OWASP API Security Top 10*)

Business Impact: An attacker could script a simple loop across sequential IDs to scrape an entire platform's content and harvest every commenter's email address without valid credentials, a direct path to mass data exposure, spam, or phishing campaigns.

Remediation Suggestions:

- Enforce server-side authorization checks that verify the requesting party has permission to view the specific object requested, not just that the ID is well-formed.
- Avoid returning commenter email addresses in public-facing endpoints; use an internal reference resolved only for authorized parties.
- Apply consistent object-level access control across all resource and sub-resource endpoints.

Finding 4: Proper Authentication Enforcement and Data Minimization

API Tested: ReqRes

Endpoint(s): `GET /api/users` (no key vs. valid x-api-key)

Test Performed: Compared behavior with no x-api-key header (rejected) against a request with a valid key (accepted).

Result: Unauthenticated requests were correctly rejected with a 401 Unauthorized and a clear error message before any data was returned. Authenticated requests returned only id, email, first_name, last_name, and avatar; no addresses, phone numbers, or other sensitive fields.

The screenshot shows a web browser interface with the URL `https://reqres.in/api/users`. The status bar indicates a **401 Unauthorized** response. The response body is displayed in JSON format, showing an error message and instructions for authentication.

```
{
  "error": "missing_api_key",
  "message": "The x-api-key header is required for this endpoint.",
  "hint": "Create a free key at app.reqres.in and send it as x-api-key.",
  "next_steps": [
    "Go to app.reqres.in/api-keys to get your key",
    "Add header: x-api-key: <your_key>",
    "For app user endpoints, use Authorization: Bearer <session_token> instead"
  ],
  "docs_url": "https://app.reqres.in/docs#authentication",
  "example_curl": "curl -H \"x-api-key: YOUR_API_KEY\" https://api.reqres.in/api/collections",
  "_meta": {
    "powered_by": "ReqRes",
    "docs_url": "https://app.reqres.in/documentation",
    "upgrade_url": "https://app.reqres.in/upgrade",
    "example_url": "https://app.reqres.in/examples/notes-app",
    "variant": "v1_b",
    "message": "Missing/invalid key. Grab a free key to keep responses consistent.",
    "cta": {}
  }
}
```

The screenshot shows a web browser interface with the URL `https://reqres.in/api/users?page=2`. The status bar indicates a **200 OK** response. The response body is displayed in JSON format, showing a list of users on page 2.

```
{
  "page": 2,
  "per_page": 6,
  "total": 12,
  "total_pages": 2,
  "data": [
    {
      "id": 7,
      "email": "michael.lawson@reqres.in",
      "first_name": "Michael",
      "last_name": "Lawson",
      "avatar": "https://reqres.in/img/faces/7-image.jpg"
    },
    {
      "id": 8,
      "email": "lindsay.ferguson@reqres.in",
      "first_name": "Lindsay",
      "last_name": "Ferguson",
      "avatar": "https://reqres.in/img/faces/8-image.jpg"
    }
  ]
}
```

Risk Category: Authentication

Severity: Pass

Business Impact: None, this is a positive control. It demonstrates baseline access control combined with data minimization, and serves as a useful contrast to the unauthenticated exposure identified in Finding 1.

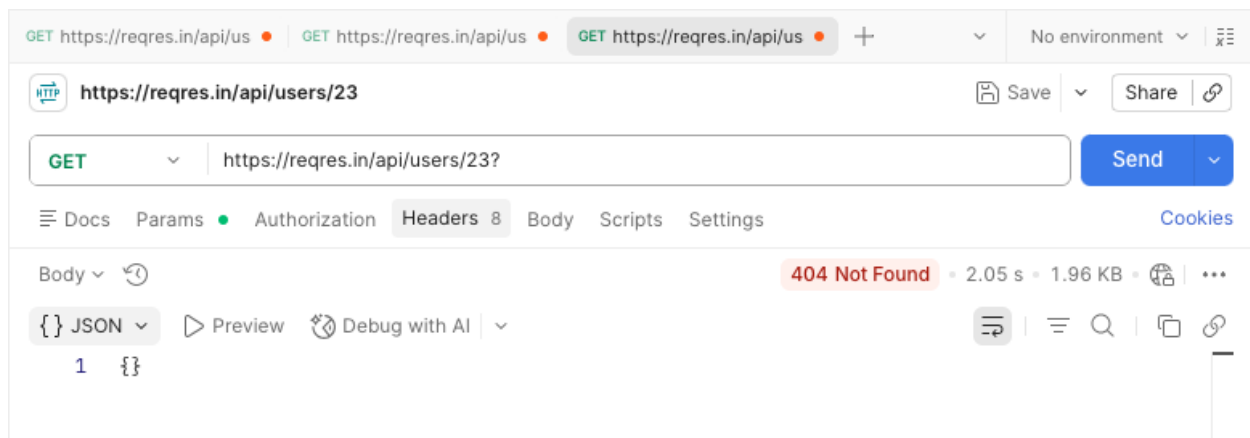
Finding 5: Safe Error Handling on Invalid IDs (Authenticated)

API Tested: ReqRes

Endpoint(s): GET /api/users/23 (with valid x-api-key)

Test Performed: Requested a user ID outside the known dataset while authenticated.

Result: Returned a clean 404 Not Found with an empty JSON body, consistent with JSONPlaceholder's behavior.



Risk Category: Input Validation

Severity: Pass

Business Impact: None, this is a positive control, consistent with good practice observed on the other tested API.

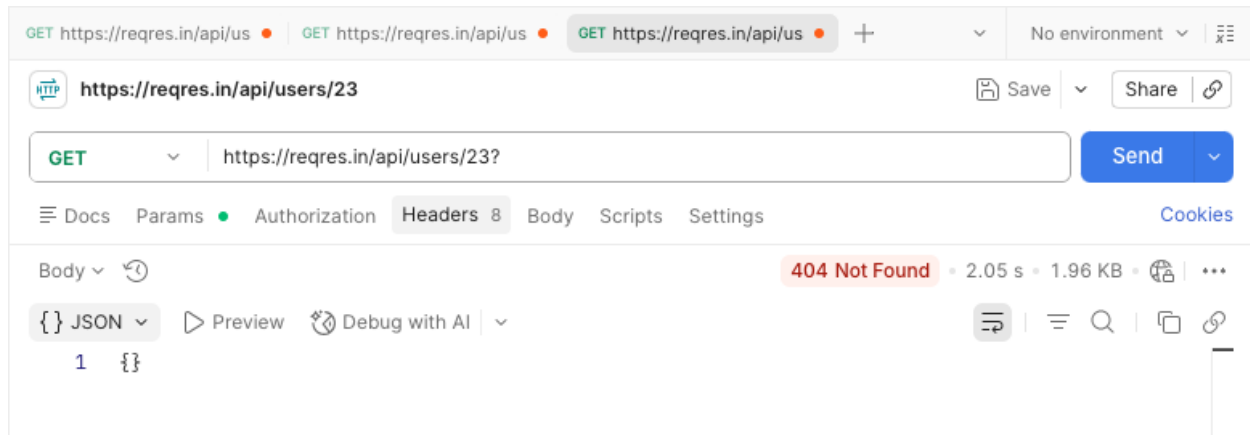
Finding 6: Missing Rate Limiting

API Tested: ReqRes

Endpoint(s): GET /api/users/23 (repeated in quick succession)

Test Performed: Sent the same authenticated request repeatedly and rapidly to check for throttling behavior.

Result: No rate limiting was observed. Every request returned normally with consistent response times (~500ms) and no 429 Too Many Requests, delay, or lockout, even under a single authenticated key.



Risk Category: Missing Rate Limiting

Severity: **Medium** (*would be more severe at production scale/traffic*)

Business Impact: Without rate limiting, a single valid or leaked API key could be used to scrape, enumerate, or overload the service with no built-in defense, increasing both data-exposure risk and operational cost/downtime risk for the API owner.

Remediation Suggestions:

- Implement per-key rate limiting (e.g., a fixed number of requests per minute) with 429 responses once exceeded.
- Document rate limits and recommended backoff behavior for API consumers.
- Monitor and alert on abnormal request volume per key to detect abuse in real time.

6. Comparative Observations

Testing both APIs side by side highlighted a clear contrast in security posture. JSONPlaceholder, designed purely as an open test dataset, applies no authentication or authorization controls anywhere, making it a useful illustration of what unrestricted API exposure looks like in practice. ReqRes, by contrast, enforces authentication and returns a minimized data set, but still lacks rate limiting even for authenticated requests. Together, the two APIs demonstrate that authentication alone does not constitute complete API security, access control, data minimization, and abuse prevention must all be addressed together.

7. Overall Risk Rating & Recommendations

Based on the findings above, the following prioritized recommendations apply broadly to both tested APIs and to any production system following similar patterns:

- **Priority 1** - Enforce object-level authorization on every resource and sub-resource endpoint, not just top-level authentication.
- **Priority 2** - Require authentication on any endpoint returning personal or sensitive data, and apply field-level data minimization.

- **Priority 3** - Implement rate limiting on all endpoints, including authenticated ones, to prevent enumeration and abuse.
- **Ongoing** - Maintain generic, non-verbose error responses to limit reconnaissance value for attackers.

8. Conclusion

This assessment used only safe, read-only, publicly permitted testing methods to evaluate two demo APIs against common API security risk categories, including several drawn directly from the OWASP API Security Top 10. The findings show a realistic mix of genuine risk (unauthenticated data exposure, broken object-level authorization, missing rate limiting) alongside sound existing practices (authentication enforcement, safe error handling). This mirrors the kind of balanced, evidence-based reporting expected from a professional API security audit.